

RAPPORT DE STAGE 2

BIRGER.

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de mon stage ainsi qu'à la réalisation de ce rapport.

Mes remerciements s'adressent tout d'abord à Birger co.ltd pour m'avoir offert l'opportunité d'effectuer ce stage au sein de sa structure. Cette expérience m'a permis de découvrir un environnement professionnel exigeant, structuré et particulièrement enrichissant, tant sur le plan technique que sur le plan humain.

Je remercie sincèrement Mr Pressley Pavaday, mon encadrant de stage, pour sa disponibilité, ses conseils, sa confiance et l'accompagnement qu'il m'a apporté tout au long de cette période. Son encadrement m'a permis de mieux comprendre les enjeux du projet et d'améliorer ma démarche de travail.

J'adresse également mes remerciements à Mr Yusuf Serally, mon tuteur académique, pour son suivi, ses orientations et son soutien dans l'élaboration de ce rapport.

Je remercie aussi l'ensemble des collaborateurs de Birger co.ltd qui m'ont accueilli avec bienveillance et qui ont contribué à rendre cette expérience très formatrice.

Enfin, je tiens à remercier ma famille, mes proches et toutes les personnes qui m'ont soutenu moralement durant cette période de formation.

Résumé

Dans le cadre de mon stage de fin de cycle de Brevet de Technicien Supérieur en Services Informatiques aux Organisations (BTS SIO), option Solutions Logicielles et Applications Métiers (SLAM), j'ai participé à la conception et au développement d'une application web nommée ChequeScanner, destinée à améliorer la gestion, le contrôle et la traçabilité des chèques dans un environnement professionnel de type bancaire ou financier.

Le projet répond à un besoin de centralisation des opérations liées aux chèques, d'automatisation de l'importation de données issues de fichiers XML bancaires au format SWIFT, et de sécurisation du processus de validation grâce à un workflow Maker/Checker. L'application permet de réduire les erreurs manuelles, d'assurer la séparation des responsabilités et de renforcer la conformité des traitements.

La solution a été développée à l'aide d'ASP.NET Core .NET 10, Entity Framework Core, SQL Server, ASP.NET Core Identity, ainsi que des technologies de présentation telles que Razor Pages, MVC, Bootstrap 5.3, CSS3 et JavaScript. Le projet a nécessité une réflexion approfondie sur l'architecture logicielle, la modélisation de la base de données, la sécurité applicative, la traçabilité et l'ergonomie.

Mots-clés : *chèque, workflow Maker/Checker, import XML, traçabilité, ASP.NET Core, SQL Server, sécurité, application web, BTS SIO.*

Abstract

As part of my final internship for the BTS SIO program (Higher Technician Certificate in IT Services for Organizations), I contributed to the design and development of a web application called ChequeScanner, intended to improve cheque management, control, and traceability in a banking or financial business environment.

The project addresses the need to centralize cheque-related operations, automate data import from banking XML files in SWIFT format, and secure the validation process through a Maker/Checker workflow. The application helps reduce manual errors, ensures separation of duties, and strengthens process compliance.

The solution was developed using ASP.NET Core .NET 10, Entity Framework Core, SQL Server, ASP.NET Core Identity, as well as frontend technologies such as Razor Pages, MVC, Bootstrap 5.3, CSS3, and JavaScript. This project required in-depth work on software architecture, database design, application security, auditability, and user experience.

Keywords: *cheque, Maker/Checker workflow, XML import, traceability, ASP.NET Core, SQL Server, security, web application.*

Table des Matières

Remerciements

Résumé / Abstract

Introduction Générale

Partie I – Présentation du cadre du stage et du projet

- 1.1. Présentation de l'organisme d'accueil — Birger co.ltd
- 1.2. Contexte général du stage
- 1.3. Problématique
- 1.4. Objectifs du stage
- 1.5. Méthodologie adoptée
- 1.6. Planning de réalisation

Partie II – Analyse des besoins et étude fonctionnelle

- 2.1. Étude de l'existant et limites du système initial
- 2.2. Analyse des besoins fonctionnels
- 2.3. Analyse des besoins non fonctionnels
- 2.4. Acteurs du système et répartition des rôles
- 2.5. Cas d'utilisation

Partie III – Conception de la solution

- 3.1. Architecture générale de l'application
- 3.2. Conception de la base de données (MCD / MLD)
- 3.3. Diagramme UML des classes
- 3.4. Conception du workflow Maker/Checker
- 3.5. Sécurité logique de la solution

Partie IV – Réalisation et implémentation technique

- 4.1. Environnement technique et stack technologique
- 4.2. Mise en place du projet ASP.NET Core
- 4.3. Module d'authentification multi-rôles
- 4.4. Module d'import XML bancaire (SWIFT)
- 4.5. Module de gestion des chèques
- 4.6. Module d'administration
- 4.7. Interface utilisateur et expérience
- 4.8. Gestion des fichiers et pièces jointes

Partie V – Tests, résultats, difficultés et perspectives

- 5.1. Stratégie de tests
- 5.2. Cas de test fonctionnels
- 5.3. Résultats obtenus
- 5.4. Difficultés rencontrées et solutions apportées
- 5.5. Bilan du stage et compétences développées
- 5.6. Perspectives d'évolution

Conclusion Générale

Bibliographie et Webographie

Annexes

Introduction Générale

Dans un environnement bancaire et financier, la gestion des chèques demeure une activité sensible nécessitant rigueur, traçabilité et sécurité. Même si les moyens de paiement électroniques se sont largement développés ces dernières années, le chèque continue d'être utilisé dans de nombreux contextes professionnels à l'échelle locale et internationale. Son traitement implique des étapes multiples : réception, enregistrement, contrôle, validation, archivage et suivi.

Lorsqu'il est réalisé de manière partiellement manuelle ou à l'aide d'outils dispersés, ce processus devient source d'erreurs, de lenteurs et de difficultés de contrôle. Le projet ChequeScanner s'inscrit dans cette problématique. Il vise à proposer une application web centralisée permettant de gérer les chèques, de tracer l'ensemble des opérations associées et de structurer leur validation au moyen d'un workflow Maker/Checker garantissant la séparation des responsabilités.

Ce stage, réalisé au sein de Birger co.ltd dans le cadre de ma formation BTS SIO option SLAM, m'a permis d'aborder un projet de développement d'entreprise complet, depuis la phase d'analyse initiale jusqu'à la livraison d'une solution fonctionnelle et testée. À travers ce projet, il s'agissait de répondre à un besoin métier concret en mobilisant des compétences techniques avancées en développement web, en architecture logicielle, en sécurité applicative et en modélisation de bases de données.

Le présent rapport décrit l'ensemble du travail réalisé au cours du stage. Il présente tout d'abord le cadre général du stage ainsi que le contexte du projet, puis expose l'analyse des besoins, la conception de la solution, la réalisation technique, les tests effectués, les difficultés rencontrées et les perspectives d'évolution envisageables.

Partie I – Présentation du cadre du stage et du projet

1.1. Présentation de l'organisme d'accueil — Birger co.ltd

Le stage s'est déroulé au sein de Birger co.ltd, une structure spécialisée dans les services financiers et les solutions bancaires. L'entreprise évolue dans un environnement où la fiabilité des données, la sécurité des systèmes d'information et le respect des procédures internes constituent des exigences fondamentales.

L'organisation accueille différents profils métier et techniques, notamment des administrateurs, des opérateurs de saisie, des responsables de validation et des équipes de conformité. Dans ce contexte, les applications internes jouent un rôle central dans la fluidité des opérations et la maîtrise des risques opérationnels.

1.2. Contexte général du stage

Le stage s'inscrivait dans une démarche d'amélioration des outils internes de gestion de documents et d'opérations financières. Le besoin exprimé par Birger co.ltd concernait plus précisément le traitement des chèques, notamment :

- L'importation automatisée de données depuis des fichiers XML bancaires au format SWIFT
- La centralisation des informations relatives aux chèques dans une base de données unique
- La séparation formelle des responsabilités entre créateur (Maker) et validateur (Checker)
- La traçabilité complète des actions effectuées sur chaque chèque
- L'administration sécurisée des comptes utilisateurs et des droits d'accès

Le projet ChequeScanner a ainsi été conçu comme une application web d'entreprise orientée vers la gestion et le suivi du cycle de vie complet d'un chèque.

1.3. Problématique

Dans de nombreuses organisations, le traitement des chèques s'appuie encore sur des procédures semi-manuelles ou peu centralisées. Cela entraîne plusieurs difficultés majeures : risque élevé d'erreurs de saisie, absence de visibilité globale sur les traitements en cours, difficulté à retracer l'historique des opérations, faiblesse du contrôle interne et complexité des audits.

Problématique : *Comment concevoir une application web sécurisée, centralisée et traçable permettant de gérer les chèques, d'importer des données XML bancaires et d'appliquer un workflow Maker/Checker conforme aux exigences métier ?*

1.4. Objectifs du stage

L'objectif principal était de développer une application web nommée ChequeScanner répondant aux contraintes techniques et métier du traitement des chèques. Plus précisément :

- Mettre en place une architecture applicative moderne et évolutive (ASP.NET Core .NET 10)
- Concevoir une base de données relationnelle robuste et normalisée (SQL Server)

- Intégrer un mécanisme d'authentification et d'autorisation multi-rôles (Identity)
- Développer un moteur d'import de fichiers XML bancaires (SWIFT / LINQ to XML)
- Implémenter un workflow de validation Maker/Checker avec gestion des statuts
- Proposer une interface utilisateur sécurisée, ergonomique et responsive (Bootstrap 5.3)
- Garantir la traçabilité complète des opérations et des modifications

1.5. Méthodologie adoptée

La réalisation du projet a été conduite de manière progressive et structurée, selon les étapes suivantes : analyse du besoin métier et définition des spécifications, modélisation des données et conception de l'architecture technique, développement itératif des modules, intégration des composants, tests fonctionnels et validation, puis documentation finale. Cette méthodologie a permis d'avancer de manière cohérente tout en gardant une certaine flexibilité pour ajuster les choix techniques en cours de réalisation.

1.6. Planning de réalisation

Semaine	Activité principale
1	Prise en main de l'environnement de développement et compréhension du besoin métier
2	Analyse fonctionnelle et rédaction des spécifications techniques
3	Modélisation de la base de données (MCD, MLD)
4	Mise en place du projet ASP.NET Core .NET 10 et configuration de l'environnement
5	Intégration d'ASP.NET Core Identity et gestion des rôles utilisateurs
6	Développement du module d'import XML (fichiers bancaires SWIFT)
7	Développement du module de gestion des chèques (CRUD, filtres, recherche)
8	Implémentation du workflow Maker/Checker (Draft → Pending → Approved/Rejected)
9	Développement du module d'administration (gestion des utilisateurs, logs)
10	Intégration de l'interface utilisateur Bootstrap 5.3 et des pièces jointes
11	Tests fonctionnels, corrections de bugs, documentation finale

Partie II – Analyse des besoins et étude fonctionnelle

2.1. Étude de l'existant et limites du système initial

Avant le démarrage du projet, j'ai mené une analyse des pratiques en place au sein de Birger co.ltd concernant le traitement des chèques. Cette étude a révélé un besoin criant de modernisation et d'automatisation, le système existant présentant plusieurs lacunes importantes.

Données dispersées Informations réparties entre documents bureautiques et échanges manuels, sans centralisation ni base de données commune.	Ressaisie manuelle fastidieuse Opérations de saisie répétitives augmentant le risque d'erreur et le temps de traitement des dossiers.
Absence de contrôle formalisé Validations peu structurées, sans workflow ni séparation claire des responsabilités entre créateur et validateur.	Traçabilité insuffisante Impossibilité de retracer facilement l'historique des opérations, compliquant les audits et la conformité.

2.2. Analyse des besoins fonctionnels

Les besoins fonctionnels ont été recueillis lors d'entretiens avec les équipes métier de Birger co.ltd et ont été formalisés en spécifications. L'application doit notamment :

- Permettre l'importation automatisée de chèques depuis des fichiers XML bancaires
- Offrir une interface de saisie et de gestion manuelle des chèques (CRUD)
- Implémenter un workflow de validation à deux niveaux (Maker/Checker)
- Gérer les statuts des chèques : Draft, Pending, Approved, Rejected
- Permettre l'ajout de pièces jointes (documents associés aux chèques)
- Fournir un module d'administration pour la gestion des utilisateurs
- Assurer une recherche et un filtrage avancé sur la liste des chèques
- Journaliser toutes les actions sensibles pour des fins d'audit

2.3. Analyse des besoins non fonctionnels

- Sécurité : authentification robuste, protection CSRF, XSS, injections SQL
- Performance : temps de réponse acceptable même avec un volume important de données
- Ergonomie : interface intuitive, accessible aux utilisateurs non techniques
- Maintenabilité : code structuré, documenté et évolutif
- Compatibilité : fonctionnement sur tous les navigateurs modernes (desktop et mobile)
- Fiabilité : transactions atomiques pour garantir la cohérence des données

2.4. Acteurs du système et répartition des rôles

Rôle	Permissions et responsabilités
ITAdmin	Administration complète du système : gestion des utilisateurs, attribution des rôles, accès aux logs d'audit, configuration globale de l'application.

Maker	Création et importation des chèques, saisie des données, soumission pour validation, modification des entrées en statut Draft, gestion des pièces jointes.
Checker	Consultation des chèques en attente (Pending), validation (Approved) ou rejet avec commentaire obligatoire (Rejected), aucun droit de modification des données.

2.5. Cas d'utilisation

Les principaux cas d'utilisation identifiés sont : l'authentification et la gestion de session, l'import XML d'un fichier bancaire, la création et la modification d'un chèque, la soumission d'un chèque pour validation, l'approbation ou le rejet d'un chèque avec commentaire, la consultation de l'historique et des logs, et l'administration des comptes utilisateurs. Chaque cas d'utilisation est soumis au contrôle d'accès par rôle, garantissant que seuls les utilisateurs autorisés peuvent effectuer les opérations correspondantes.

Partie III – Conception de la solution

3.1. Architecture générale de l'application

L'application ChequeScanner repose sur une architecture en couches (N-tiers) structurée autour du pattern MVC (Model-View-Controller) propre à ASP.NET Core. Cette architecture garantit la séparation des préoccupations, la testabilité et la maintenabilité du code.

- Couche Présentation : Razor Pages, MVC Views, Bootstrap 5.3, JavaScript — gestion de l'affichage
- Couche Métier (Business Logic) : Services, validations, règles du workflow, gestion des rôles
- Couche Données : Entity Framework Core, DbContext, LINQ — accès et manipulation des données
- Base de données : SQL Server — tables métier, tables Identity, journaux d'audit

3.2. Conception de la base de données (MCD / MLD)

La conception de la base de données repose sur une modélisation entité-association garantissant l'intégrité des données et la cohérence des relations. Les principales entités du système sont les suivantes :

- Utilisateurs : gestion des comptes, rôles et informations personnelles avec contrôle d'accès granulaire (extension d'ASP.NET Core Identity)
- Chèques : données principales — référence, montant, devise, date, bénéficiaire, statut de validation, créateur, validateur
- Lots (Batches) : regroupement des chèques par import, avec comptage et traçabilité des traitements
- Messages XML : conservation des données brutes extraites des fichiers bancaires au format SWIFT
- Pièces jointes : stockage et gestion des documents associés aux chèques traités
- Journal d'audit : historisation de chaque action sensible (qui, quoi, quand)

3.3. Diagramme UML des classes

Le diagramme UML des classes reflète les entités principales du domaine et leurs relations. Les classes Cheque, Lot, XmlMessage, Utilisateur et PieceJointe constituent le cœur du modèle métier. La classe Cheque est centrale : elle est liée à un Lot (créé lors de l'import), à l'utilisateur créateur (Maker) et à l'utilisateur validateur (Checker), ainsi qu'à une collection de PiecesJointes. Les statuts sont gérés via une énumération (ChequeStatus : Draft, Pending, Approved, Rejected).

3.4. Conception du workflow Maker/Checker

Le workflow de validation suit un cycle de vie strict à quatre états, garantissant la séparation des responsabilités :

- Draft : état initial après création ou import. Seul le Maker peut modifier un chèque dans cet état.
- Pending : après soumission par le Maker. Le chèque est verrouillé en modification et attend la décision du Checker.
- Approved : le Checker a validé le chèque. Le chèque devient définitivement non modifiable.

- Rejected : le Checker a rejeté le chèque avec un commentaire obligatoire. Le Maker peut corriger et resoumettre.

Les règles métier imposent qu'un Maker ne puisse jamais valider ses propres saisies, et qu'un Checker ne puisse pas créer de chèques. Un chèque approuvé est immuable. Toutes les transitions de statut sont enregistrées dans le journal d'audit.

3.5. Sécurité logique de la solution

La sécurité de l'application est assurée par plusieurs mécanismes complémentaires :

- Authentification native via ASP.NET Core Identity avec contrôle de session et protection anti-CSRF intégrée
- Autorisation granulaire par rôle (ITAdmin, Maker, Checker) avec attributs [Authorize(Roles="...")]
- Validation serveur systématique de toutes les entrées utilisateur (data annotations, model binding)
- Protection contre les injections SQL grâce à Entity Framework Core et aux requêtes paramétrées
- Hachage sécurisé des mots de passe (PBKDF2 avec salt, géré par Identity)
- Journalisation des actions sensibles pour des fins d'audit et de conformité

Partie IV – Réalisation et implémentation technique

4.1. Environnement technique et stack technologique

L'ensemble des choix technologiques a été guidé par les exigences de sécurité, de performance, de maintenabilité et d'évolutivité de l'application.

Catégorie	Technologie	Rôle
Backend	ASP.NET Core .NET 10 / C#	Framework principal de l'application
ORM	Entity Framework Core	Accès aux données et migrations BD
Base de données	Microsoft SQL Server	Stockage relationnel des données
Authentification	ASP.NET Core Identity	Gestion des utilisateurs et des rôles
Frontend	Razor Pages / MVC	Génération des vues côté serveur
CSS / UI	Bootstrap 5.3 / CSS3	Interface responsive et ergonomique
Scripts	JavaScript	Interactions dynamiques côté client
Import bancaire	LINQ to XML	Parsing des fichiers XML SWIFT
IDE	Visual Studio 2022	Environnement de développement
Versioning	Git / GitHub	Gestion du code source

4.2. Mise en place du projet ASP.NET Core

Le projet a été initialisé sous Visual Studio 2022 en utilisant le template ASP.NET Core Web App (MVC) avec authentification intégrée. La configuration initiale a inclus l'ajout d'Entity Framework Core avec SQL Server, la mise en place des migrations pour la création automatique du schéma de base de données, et la configuration du pipeline de middlewares (authentification, autorisation, gestion des erreurs, fichiers statiques).

4.3. Module d'authentification multi-rôles

Le module d'authentification repose sur ASP.NET Core Identity, une bibliothèque mature et éprouvée qui gère nativement le hachage des mots de passe, les sessions, la gestion des rôles et la protection contre les attaques courantes (CSRF, brute force). Trois rôles ont été définis et seédés en base de données lors du démarrage de l'application : ITAdmin, Maker et Checker. Chaque contrôleur et action est protégé via l'attribut [Authorize], garantissant qu'un utilisateur ne peut accéder qu'aux fonctionnalités correspondant à son rôle.

4.4. Module d'import XML bancaire (SWIFT)

Le module d'import XML constitue l'une des fonctionnalités centrales de l'application. Il permet l'extraction automatique des données bancaires à partir de fichiers XML standards. Le processus se déroule en cinq étapes :

- Upload : l'utilisateur Maker sélectionne et soumet le fichier XML via l'interface web
- Validation du format : vérification que le fichier est un XML bien formé avant tout traitement
- Parsing via LINQ to XML : lecture structurée du document et extraction des éléments clés (référence, montant, devise, bénéficiaire, date)
- Création du lot (Batch) : génération automatique d'un enregistrement de lot avec métadonnées (date, utilisateur, nombre de chèques)
- Persistance transactionnelle : enregistrement en base avec rollback automatique en cas d'erreur pour garantir l'intégrité des données

4.5. Module de gestion des chèques

Le module de gestion des chèques offre une interface complète pour consulter, créer, modifier, filtrer et suivre les chèques. La liste des chèques est paginée et offre des filtres par statut, par date, par montant et par bénéficiaire. Chaque chèque dispose d'une vue de détail affichant toutes les informations, l'historique des actions et les pièces jointes associées. Les actions disponibles (modifier, soumettre, valider, rejeter) sont affichées dynamiquement selon le rôle de l'utilisateur connecté et le statut actuel du chèque.

4.6. Module d'administration

Accessible uniquement à l'ITAdmin, le module d'administration permet de gérer les comptes utilisateurs (création, activation/désactivation, attribution des rôles), de consulter les journaux d'audit, et d'accéder à des statistiques globales sur l'activité de l'application (nombre de chèques par statut, volume traité par période, taux de validation/rejet).

4.7. Interface utilisateur et expérience

L'interface a été conçue avec Bootstrap 5.3 pour garantir une expérience cohérente et adaptée à tous les profils utilisateurs et sur tous les appareils (desktop, tablette, mobile). Les pages principales sont : la page de connexion sécurisée, le tableau de bord avec statistiques, la liste des chèques avec badges de statut colorés, et les formulaires de saisie et de validation avec messages de confirmation clairs. L'ergonomie a été une priorité : navigation intuitive, retours visuels immédiats, formulaires validés côté client et côté serveur.

4.8. Gestion des fichiers et pièces jointes

L'application permet d'associer des documents (images, PDFs, justificatifs) à chaque chèque. Les fichiers sont téléversés de manière sécurisée, leur type est vérifié côté serveur, et ils sont stockés dans un répertoire dédié sur le serveur avec un nom de fichier unique généré pour éviter les collisions. Les métadonnées (nom original, type MIME, date d'upload, utilisateur) sont enregistrées en base de données.

Partie V – Tests, résultats, difficultés et perspectives

5.1. Stratégie de tests

Une stratégie de tests complète a été mise en place pour valider le fonctionnement de l'application avant la livraison. Les tests ont été conduits en trois catégories : tests fonctionnels (validation des scénarios métier), tests de non-régression (vérification qu'une correction n'introduit pas de nouveaux bugs), et tests de sécurité (tentatives d'accès non autorisés, injection de données malveillantes). Les tests ont été exécutés manuellement sur l'ensemble des modules.

5.2. Cas de test fonctionnels

Cas de test	Résultat attendu	Statut	Module
Connexion avec identifiants valides	Accès accordé, redirection selon le rôle	✓ Validé	Authentification
Connexion avec identifiants invalides	Message d'erreur, accès refusé	✓ Validé	Authentification
Import d'un fichier XML valide	Lot créé, chèques enregistrés en Draft	✓ Validé	Import XML
Import d'un fichier XML malformé	Erreur affichée, aucune donnée enregistrée	✓ Validé	Import XML
Soumission d'un chèque pour validation	Statut passe de Draft à Pending	✓ Validé	Workflow
Validation par un Checker autorisé	Statut passe à Approved	✓ Validé	Workflow
Rejet sans commentaire	Rejet bloqué, message d'obligation	✓ Validé	Workflow
Rejet avec commentaire	Statut passe à Rejected, commentaire enregistré	✓ Validé	Workflow
Modification d'un chèque Approved	Action refusée, chèque en lecture seule	✓ Validé	Sécurité
Accès Maker à la validation	Accès refusé (403), séparation des rôles	✓ Validé	Sécurité
Ajout d'une pièce jointe	Fichier enregistré et lié au chèque	✓ Validé	Pièces jointes
Création d'un utilisateur (ITAdmin)	Utilisateur créé avec le rôle assigné	✓ Validé	Administration

5.3. Résultats obtenus

L'ensemble des cas de test fonctionnels ont été validés avec succès. Le taux de réussite global est de 100% sur les scénarios nominaux et les cas d'erreur testés. Les performances de l'application sont satisfaisantes, avec des temps de réponse inférieurs à une seconde pour les opérations courantes. L'interface est fonctionnelle et claire sur les différents navigateurs testés (Chrome, Firefox, Edge) et sur les différentes tailles d'écran.

5.4. Difficultés rencontrées et solutions apportées

Difficultés techniques

La principale difficulté technique a été la gestion des transactions lors de l'import XML. En cas d'erreur partielle (fichier contenant des données invalides), il fallait garantir que soit la totalité des chèques était importée, soit aucun. L'utilisation des transactions Entity Framework Core (BeginTransaction / Rollback) a résolu ce problème, assurant l'intégrité des données.

La gestion fine des autorisations dans le contexte du workflow a également été un défi : il fallait vérifier non seulement le rôle de l'utilisateur, mais aussi l'état actuel du chèque avant d'autoriser chaque action. La mise en place de services dédiés avec des méthodes de validation métier a permis de centraliser et de tester cette logique.

Difficultés organisationnelles

La phase d'analyse des besoins a nécessité plusieurs échanges avec l'équipe métier de Birger co.ltd pour bien comprendre les subtilités du workflow Maker/Checker et les règles métier associées. La mise en place d'une communication régulière, de réunions de suivi hebdomadaires et de démonstrations intermédiaires a permis de valider les développements au fur et à mesure et d'éviter des reprises coûteuses.

5.5. Bilan du stage et compétences développées

Ce stage a été une expérience de formation particulièrement enrichissante, me permettant de consolider et d'approfondir des compétences clés :

- Développement web full-stack avec ASP.NET Core .NET 10 et Entity Framework Core
- Conception et modélisation de bases de données relationnelles (MCD, MLD, migrations EF)
- Architecture logicielle d'entreprise : MVC, injection de dépendances, services métier
- Sécurité applicative : Identity, autorisation par rôle, protection CSRF/XSS
- Traitement et parsing de fichiers XML bancaires avec LINQ to XML
- Gestion de projet : structuration des tâches, rédaction de spécifications, communication avec l'équipe
- Rigueur et méthode : tests fonctionnels, documentation technique et rédaction de rapport

5.6. Perspectives d'évolution

Le projet ChequeScanner constitue une base solide pour de nombreuses évolutions futures :

- Module d'audit détaillé : historisation complète avec analyse comparative et reporting avancé (export Excel/PDF)
- Moteur OCR intégré : extraction automatique depuis des images de chèques scannés avec reconnaissance optique
- Notifications par email : alertes automatiques lors des validations, rejets ou anomalies détectées
- Tableau de bord statistique : métriques de performance, KPIs de traitement et visualisations interactives
- API REST exposée : intégration avec d'autres systèmes bancaires et applications tierces pour l'interopérabilité

- Déploiement Cloud CI/CD : infrastructure évolutive avec pipeline de déploiement continu

Conclusion Générale

Le projet ChequeScanner constitue une solution complète répondant aux besoins de gestion, de contrôle et de traçabilité des opérations sur chèques dans un environnement professionnel exigeant. Développée au sein de Birger co.ltd dans le cadre de mon stage de fin de cycle BTS SIO, cette application web a mobilisé l'ensemble des compétences techniques et transversales acquises au cours de ma formation.

Sur le plan technique, ce projet m'a permis d'approfondir ma maîtrise d'ASP.NET Core .NET 10, d'Entity Framework Core, de SQL Server et des bonnes pratiques de sécurité applicative. La conception et l'implémentation du workflow Maker/Checker ont en particulier été une expérience formatrice, car elles nécessitaient de concilier des exigences métier précises avec des contraintes techniques complexes.

Sur le plan personnel, ce stage m'a renforcé dans ma capacité à mener un projet de développement de A à Z — de l'analyse des besoins à la livraison d'une solution testée et documentée — tout en développant des qualités essentielles comme l'autonomie, la rigueur, l'organisation et la communication en milieu professionnel.

ChequeScanner représente une base solide pour une solution moderne, sécurisée et évolutive, ouvrant la voie à l'automatisation avancée et à l'intégration avec l'écosystème bancaire. Ce mémoire marque la fin de cette phase de formation, mais constitue le point de départ d'un projet professionnel prometteur et d'une carrière dans le développement d'applications d'entreprise.

Bibliographie et Webographie

Ouvrages de référence

- A. LOCK, ASP.NET Core in Action, 3rd edition, Manning Publications, 2023.
- M. FOWLER, Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.
- C. MARTIN (Uncle Bob), Clean Architecture, Prentice Hall, 2017.

Documentation technique officielle

- Documentation officielle ASP.NET Core : <https://docs.microsoft.com/fr-fr/aspnet/core/>
- Documentation Entity Framework Core : <https://docs.microsoft.com/fr-fr/ef/core/>
- Documentation ASP.NET Core Identity : <https://docs.microsoft.com/fr-fr/aspnet/core/security/authentication/identity>
- Documentation Bootstrap 5.3 : <https://getbootstrap.com/docs/5.3/>
- Documentation SWIFT XML Standards : <https://www.swift.com/standards>

Ressources en ligne

- Microsoft Learn — Parcours de formation .NET : <https://learn.microsoft.com/fr-fr/dotnet/>
- Stack Overflow — Résolution de problèmes techniques : <https://stackoverflow.com/>
- GitHub — Gestion du code source et références de projets : <https://github.com/>

Annexes

Annexe A — Liste des figures

- Figure 1 : Démarche générale du projet
- Figure 2 : Architecture générale de l'application ChequeScanner
- Figure 3 : Diagramme de contexte du système
- Figure 4 : Workflow Maker/Checker (diagramme d'états)
- Figure 5 : Diagramme de cas d'utilisation
- Figure 6 : Modèle Conceptuel de Données (MCD)
- Figure 7 : Modèle Logique de Données (MLD)
- Figure 8 : Diagramme UML des principales classes
- Figure 9 : Flux de traitement de l'import XML
- Figure 10 : Schéma d'authentification et d'autorisation

Annexe B — Liste des tableaux

- Tableau 1 : Planning prévisionnel du stage
- Tableau 2 : Répartition des rôles utilisateurs
- Tableau 3 : Stack technologique utilisée
- Tableau 4 : Cas de test fonctionnels et résultats

Annexe C — Liste des sigles et abréviations

- API : Application Programming Interface
- ASP.NET : Active Server Pages .NET
- CRUD : Create, Read, Update, Delete
- CSRF : Cross-Site Request Forgery
- EF Core : Entity Framework Core
- MCD : Modèle Conceptuel de Données
- MLD : Modèle Logique de Données
- MVC : Model View Controller
- ORM : Object Relational Mapping
- SLAM : Solutions Logicielles et Applications Métiers
- SQL : Structured Query Language
- SWIFT : Society for Worldwide Interbank Financial Telecommunication
- UML : Unified Modeling Language
- XML : eXtensible Markup Language
- XSS : Cross-Site Scripting